

Deep Residual Learning for Image Recognition

Anonymous CVPR submission

Paper ID 6666

Abstract

Deeper neural networks are more difficult to train. (开篇直接点出核心问题:深度神经网络的训练是困难的)
We present [a residual learning framework] to ease the training of networks that substantially (显著地adv.) deeper than those used previously. (①提出方法框架) *We explicitly reformulate the layers as learning residual functions with reference to the layer inputs, instead of learning unreferenced functions.* (②解释核心机制:残差学习的本质不是让网络凭空学习目标函数,而是参照层的输入来学习残差函数) *We provide comprehensive empirical (实证地adj.) evidence showing that these residual networks are easier to optimize, and can gain accuracy from considerably (显著地adv.) increased depth.* (③实验支撑并总结效果) *On ImageNet dataset we evaluate residual nets with a depth of up to 152 layers – $8\times$ deeper than VGG nets but still having lower complexity. An ensemble (集成n.) of these residual nets achieves 3.57% error on ImageNet test set.* (④展示效果) *This result won the 1st place on the ILSVRC 2015 classification task. We also present analysis on CIFAR-10 with 100 and 1000 layers.* (⑤用比赛结果提高说服力)

(We present → We reformulate → We provide → We evaluate)

The depth of representations (网络层对输入数据逐步抽象后的特征表达) is of central importance for many visual recognition tasks. Solely (Only的高级用法) due to our extremely deep representations, we obtain (实现v.) a 28% relative improvement on the COCO object detection dataset. Deep residual nets are foundations of our submissions to ILSVRC & COCO 2015 competitions, where we also won the 1st places on the tasks of ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation. (进一步论证了Deep Residual Net的泛化效果)

1. Introduction

Deep convolutional neural networks have led to a series of breakthroughs for image classification. Deep networks

naturally integrate low/mid/high-level features and classifiers in an end-to-end multilayer fashion, and the "levels" of features can be enriched by the number of stacked layers (depth). Recent evidence reveals that network depth is of crucial importance, and leading results on the challenging ImageNet dataset all exploit (采用v.) "very deep" models, with a depth of sixteen to thirty. Many other nontrivial visual recognition tasks have also greatly benefited from very deep models. (网络的深度是影响视觉识别相关任务的关键,加深网络层数可以学习到图像中高层次的视觉特征,从未达到更好的分类效果)

Driven by the significance of **depth**, a question arises: *Is learning better networks as easy as stacking more layers?* (在认识到了层数的重要性后提出问题:学习更好的网络是否就像堆叠更多的层一样简单?) An obstacle to answering this question was the notorious problem of vanishing/exploding gradients, which hamper convergence from the beginning. This problem, however, has been largely **addressed** (已解决v.) by normalized initialization and intermediate normalization layers, which enable networks with tens of layers to start converging for stochastic gradient descent (SGD) with backpropagation. (权重随机初始化可以帮助缓解梯度爆炸/消失)

When deeper networks are able to start converging, a **degradation** (性能退化adj.) problem has been exposed: with the network depth increasing, accuracy gets saturated (which might be unsurprising) and then degrades rapidly. (随机梯度的反向传播训练可以使深层的网络收敛,但是随着网络层数加深,准确率会达到饱和并迅速退化) Unexpectedly, such **degradation is not caused by overfitting**, and **adding more layers to a suitably deep models leads to higher training error**, as reported in [10,41] and thoroughly verified by our experiments, Fig.1 shows a typical example. (对于一个深度适中的网络,增加层数会带来更高的训练误差[文献10、41以及文本的实验中证实了这个问题])

The degradation (of training accuracy) indicates that not all systems are similarly easy to optimize. (如果是过拟合,那么训练误差应该仍然很好;训练误差本身变差,说明问题出在优化上,网络深了但是SGD优化算法找不到更好的解) Let us consider a shallower architecture and

its deeper counterpart that adds more layers onto it. There exists a solution by construction to the deeper model: the added layers are identity mapping, and the other layers are copied from the learned shallower model. (对于深层网络而言,存在一个构造解:把浅网络学好的权重原样拷进前几层,新增的层设为恒等映射) The existence of this constructed solution indicates that a deeper model should produce no higher training error than its shallower counterpart. (既然这样的解存在,深网络的训练误差理应不高于浅网络) But experiments show that our current solvers on hand **unable to find solutions** that are comparably good or better than the constructed solution (or unable to do so in feasible time). (但实验表明,我们手头的求解器找不到与构造解相当或更好的)

In this paper, we address the degradation problem by introducing a *deep residual learning* framework. (提出方法) Instead of hoping each few stacked layers directly fit a desired underlying mapping, we explicitly let these layers fit a residual mapping. (目标转变:拟合底层映射到拟合残差映射) Formally, denoting the desired underlying mapping as $\mathcal{H}(\mathbf{x})$, we let the stacked nonlinear layers fit another mapping of $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$. The original mapping is recast into $\mathcal{F}(\mathbf{x}) + \mathbf{x}$. **We hypothesize** that it is easier to optimize the residual mapping than to optimize the original, unreferenced mapping. **To the extreme**, if an identity mapping were optimal, it would be easier to push the residual to zero than to fit an identity mapping by a stack of nonlinear layers. (分析了特殊情况下的适用性)

The formulation of $\mathcal{F}(\mathbf{x}) + \mathbf{x}$ can be realized by feedforward neural networks with "shortcut connections" (Fig.2). Shortcut connections [2,33,48] are those skipping one or more layers. In our case, the shortcut connections simply perform *identity* mapping, and their outputs are added to the outputs of the stacked layers (Fig.2). (解释方法的具体实现) **Identity shortcut connections add neither extra parameter nor computational complexity.** The entire network can still be trained end-to-end by SGD with back-propagation, and can be easily implemented using common libraries (e.g., Caffe[19]) without modifying the solvers. (解释方法的优势)

We present comprehensive experiments on ImageNet to show the degradation problem and evaluate our method. We show that: 1) Our extremely deep residual nets are easy to optimize, but the counterpart "plain" nets (that simply stack layers) exhibit higher training error when the depth increases; 2) Our depth residual nets can easily enjoy accuracy gains from greatly increased depth, producing results substantially (显著地adv.) better than previous networks. (容易优化、层数加深不易退化)

Similar phenomena are also shown on the CIFAR-10 set, suggesting that optimization difficulties and the effects of our method **are not just akin** a particular dataset. (多数数据集泛化性证明模型的效果) We present successfully

trained models on this dataset with over 100 layers, and explore models with over 1000 layers.

On the ImageNet classification dataset [35], we obtain excellent results by extremely deep residual nets. Our 152 layers residual net is the deepest network ever presented on ImageNet, while still having lower complexity than VGG nets [40]. Our ensemble has **3.57%** top-5 error on the ImageNet *test set*, and *won the 1st place in the ILSVRC 2015 classification competition*. The extremely deep representations also have excellent generalization performance on other recognition tasks, and lead us to further *win the 1st places on: ImageNet detection, ImageNet localization, COCO detection, and COCO segmentation* in ILSVRC & COCO 2015 competitions. This strong evidence shows that the residual learning principle is generic, and we expect that it is applicable in other vision and non-vision problems. (通过竞赛效果证明模型的普适性)

2. Related Work

Residual Representations. In image recognition, VLAD [18] is a representation that encodes by the residual vectors with respect to a dictionary, and Fisher Vector [30] can be formulated as a probabilistic version [18] of VLAD. Both of them are powerful shallow representations for image retrieval and classification [4,48]. For vector quantization, encoding residual vectors [17] is shown to be more effective than encoding original vectors.

In low-level vision and computer graphics, for solving Partial Differential Equations (PDEs), the widely used Multigrid method [3] reformulates the system as subproblems at multiple scales, where each subproblem is responsible for the residual solution between a coarser and a finer scale. An alternative to Multigrid is hierarchical basis preconditioning [45,46], which relies on variables that represent residual vectors between two scales. It has been shown [3,45,46] that these solvers converge much faster than standard solvers that are unaware of the residual nature of the solutions. These methods suggest that a good reformulation or preconditioning can simplify the optimization.

Shortcut Connections. Practices and theories that lead to shortcut connections [2,34,49] have been studied for a long time. An early practice of training multi-layer perceptrons (MLPs) is to add a linear layer connected from the network input to the output [34,49]. In [44,24], a few intermediate layers are directly connected to auxiliary classifiers for addressing vanishing/exploding gradients. The papers of [39,38,31,47] propose methods for centering layer responses, gradients, and propagated errors, implemented by shortcut connections. In [44], an "inception" layer is composed of a shortcut branch and a few deeper branches.

Concurrent with our work, "highway networks" [42,43] present shortcut connections with gating functions [15].

These gates are data-dependent and have parameters, in contrast to our identity shortcuts that are parameter-free. When a gated shortcut is “closed” (approaching zero), the layers in highway networks represent *non-residual* functions. On the contrary, our formulation always learns residual functions; our identity shortcuts are never closed, and all information is always passed through, with additional residual functions to be learned. In addition, highway networks have not demonstrated accuracy gains with extremely increased depth (e.g., over 100 layers).

3. Deep Residual Learning

3.1. Residual Learning

Let us consider $\mathcal{H}(x)$ as an underlying mapping to be fit by **a few stacked layers (not necessarily the entire net)**, with x denoting the inputs to the first of these layers. If one hypothesizes that multiple nonlinear layers can asymptotically approximate complicated functions¹, then it is equivalent to hypothesize that they can asymptotically approximate the residual functions, i.e., $\mathcal{H}(x) - x$ (assuming that the input and output are of the same dimensions). (假设多层非线性层能渐近逼近复杂函数, 等价地它们也能渐近逼近残差函数 $\mathcal{H}(x) - x$) So rather than expect stacked layers to approximate $\mathcal{H}(x)$, we explicitly let these layers approximate a residual function $\mathcal{F}(x) := \mathcal{H}(x) - x$. The original function thus becomes $\mathcal{F}(x) + x$. Although both forms should be able to asymptotically approximate the desired functions (as hypothesized), the ease of learning might be different. ($\mathcal{H}(x)$ 是待拟合的基础映射)

This reformulation is motivated by the **counterintuitive phenomena (反直觉现象)** about the degradation problem (Fig.1, left). As we discussed in the introduction, if the added layers can be constructed as identity mappings, a deeper model should have training error no greater than its shallower counterpart. The degradation problem suggests that the solvers might have difficulties in approximating identity mappings by multiple nonlinear layers. With the residual learning reformulation, if identity mappings are optimal, the solvers may simply drive the weights of the multiple nonlinear layers toward zero to approach identity mappings.

In real cases, it is unlikely that identity mappings are optimal, but our reformulation may help to precondition the problem. If the optimal function is closer to an identity mapping than to a zero mapping, it should be easier for the solver to find the perturbations with reference to an identity mapping, than to learn the function as a new one. We show by experiments (Fig.7) that the learned residual functions in general have small responses, suggesting that identity mappings provide reasonable preconditioning. (“网络能逼近任意复杂函数”这个假设成立的条件下, 逼近 $\mathcal{H}$ 和逼近

$\mathcal{H} - x$ 在表达能力上没有差别。)

3.2. Identity Mapping by Shortcuts

We adopt residual learning to every few stacked layers. A building block is shown in Fig.2. Formally, in this paper we consider a building block defined as:

$$y = \mathcal{F}(x, \{W_i\}) + x. \quad (1)$$

Here x and y are the input and output vectors of the layers considered. The $\mathcal{F}(x, \{W_i\})$ represents the residual mapping to be learned. For the example, in Fig.2 that has two layers, $\mathcal{F} = W_2\sigma(W_1x)$ in which σ denotes ReLU and the biases are omitted for simplifying notations. The operation $\mathcal{F} + x$ is performed by a shortcut connection and element-wise addition. We adopt the second nonlinearity after the addition (i.e., $\sigma(y)$, see Fig.2).

The shortcut connections in Eqn.(1) introduce **neither extra parameter nor computation complexity**. This is not only attractive in practice but also important in our comparisons between plain and residual networks. We can fairly compare plain/residual networks that simultaneously have the same number of parameters, depth, width, and computational cost (except for the negligible element-wise addition). (shortcut 是恒等映射, 直接取 F 的输入 x 送到加法处)

The dimensions of x and $\mathcal{F}$ must be equal in Eqn.(1). If this is not the case (e.g., when changing the input/output channels), we can perform a linear projection W_s by the shortcut connections to match the dimensions:

$$y = \mathcal{F}(x, \{W_i\}) + W_s x \quad (2)$$

We can also use a square matrix W_s in Eqn.(1). But we will show by experiments that the identity mapping is sufficient for addressing the degradation problem and is economical, and thus W_s is only used when matching dimensions.

The form of the residual function $\mathcal{F}$ is flexible. Experiments in this paper involve a function $\mathcal{F}$ that has two or three layers (Fig.5), while more layers are possible. But if $\mathcal{F}$ has only a single layer, Eqn.(1) is similar to a linear layer: $y = W_1x + x$, for which we have not observed advantages.

We also note that although the above notations are about fully-connected layers for simplicity, they are applicable to convolutional layers. The function $\mathcal{F}(x, \{W_i\})$ can represent multiple convolutional layers. The element-wise addition is performed on two feature maps, channel by channel.

3.3. Network Architectures

We have tested various plain/residual nets, and have observed consistent phenomena. To provide instances for discussion, we describe two models for ImageNet as follows.

¹This hypothesis, however, is still an open question. See [28].

Plain Network. Our plain baselines (Fig.3, middle) are mainly inspired by the philosophy of VGG nets [40] (Fig.3, left). The convolutional layers mostly have 3×3 filters and follow two simple design rules: (i) for the same output feature map size, the layers have the same number of filters; and (ii) if the feature map size is halved, the number of filters is doubled so as to preserve the time complexity per layer. We perform downsampling directly by convolutional layers that have a stride of 2. The network ends with a global average pooling layer and a 1000-way fully-connected layer with softmax. The total number of weighted layers is 34 in Fig.3 (middle).

It is worth noticing that our model has *fewer* filters and *lower* complexity than VGG nets [40] (Fig.3, left). Our 34-layer baseline has 3.6 billion FLOPs (multiply-adds), which is only 18% of VGG-19 (19.6 billion FLOPs).

Residual Network. Based on the above plain network, we insert shortcut connections (Fig.3, right) which turn the network into its counterpart residual version. The identity shortcuts (Eqn.(1)) can be directly used when the input and output are of the same dimensions (solid line shortcuts in Fig.3). When the dimensions increase (dotted line shortcuts in Fig.3), we consider two options: (A) The shortcut still performs identity mapping, with extra zero entries padded for increasing dimensions. This option introduces no extra parameter; (B) The projection shortcut in Eqn.(2) is used to match dimensions (done by 1×1 convolutions). For both options, when the shortcuts go across feature maps of two sizes, they are performed with a stride of 2.

3.4. Implementation

Our implementation for ImageNet follows the practice in [21,40]. The image is resized with its shorter side randomly sampled in $[256, 480]$ for scale augmentation [40]. A 224×224 crop is randomly sampled from an image or its horizontal flip, with the per-pixel mean subtracted [21]. The standard color augmentation in [21] is used. We adopt batch normalization (BN) [16] right after each convolution and before activation, following [16]. We initialize the weights as in [12] and train all plain/residual nets from scratch. We use SGD with a mini-batch size of 256. The learning rate starts from 0.1 and is divided by 10 when the error plateaus, and the models are trained for up to 60×10^4 iterations. We use a weight decay of 0.0001 and a momentum of 0.9. We do not use dropout [13], following the practice in [16].

In testing, for comparison studies we adopt the standard 10-crop testing [21,40]. For best results, we adopt the fully-convolutional form as in [12], and average the scores at multiple scales (images are resized such that the shorter side is in $\{224, 256, 384, 480, 640\}$).